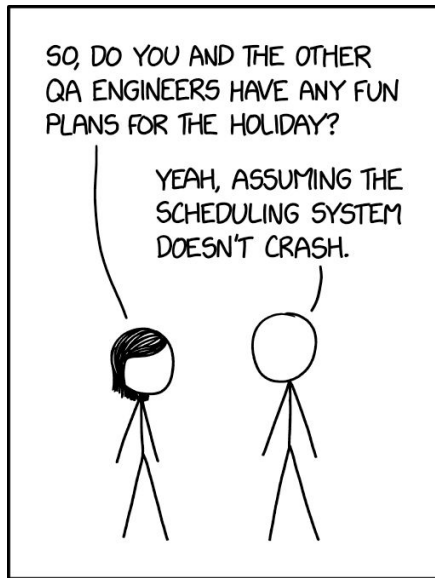


Software Engineering

Testing



SOFTWARE TESTING DAY IS A HOLIDAY
CELEBRATED EVERY -1 YEARS ON
JANUARY 0TH AT 25:71 PM.

Testing Software

If it's not tested, it doesn't work: Bugs can happen anywhere

Many types of testing, highest level division is:

Manual testing: an actual person fires everything up and starts using it

- Usability testing
- Exploratory testing
- Visual testing (can be automated, but expensive)

Automated testing: software runs through a suite of programmed tests

- Regression testing
- Unit testing
- Failure testing
- Performance testing

Regression Tests

Most common type of test

Three main categories:

Unit tests: Test a single component (one API); often use **mocks** to replace other components

Ex: prototype tests

Integration tests: Test a combination of several components, but not necessarily user-visible endpoints

Ex: Check that a server can work with an actual database

End-to-end tests: Start with a (simulated) user action, check the user-visible result, running through the whole system

Ex: Run an install script and check that servers can start up after being installed

Writing Tests: JUnit

To run a java program: Use a **main** method

To run a java test: Use an **@Test** annotation

```
import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;

public class ExampleTest {

    @Test
    public void testAddition() {
        int a = 2;
        int b = 3;
        if (a + b != 5) {
            fail();
        }
    }
}
```

Writing Tests: JUnit

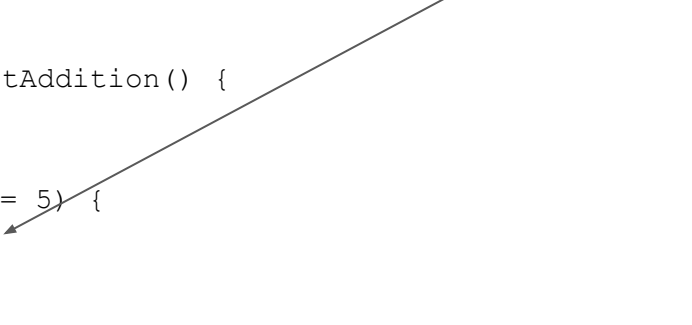
```
import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;

public class ExampleTest {

    @Test
    public void testAddition() {
        int a = 2;
        int b = 3;
        if (a + b != 5) {
            fail();
        }
    }
}
```

'static' import: Allows importing methods/constants instead of an entire class



Writing Tests: JUnit

```
import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;

public class ExampleTest {

    @Test
    public void testAddition() {
        int a = 2;
        int b = 3;
        if (a + b != 5) {
            fail();
        }
    }
}
```

JUnit 4 vs JUnit 5: to use the more up-to-date/widely supported version, make sure to import org.junit.jupiter, not regular org.junit

Writing Tests: JUnit

```
import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;

public class ExampleTest {

    @Test
    public void testAddition() {
        int a = 2;
        int b = 3;
        if (a + b != 5) {
            fail();
        }
    }
}
```

JUnit is a heavily
annotation-based
framework - **@Test** is
the most common, but
there are others (ex:
@BeforeEach)

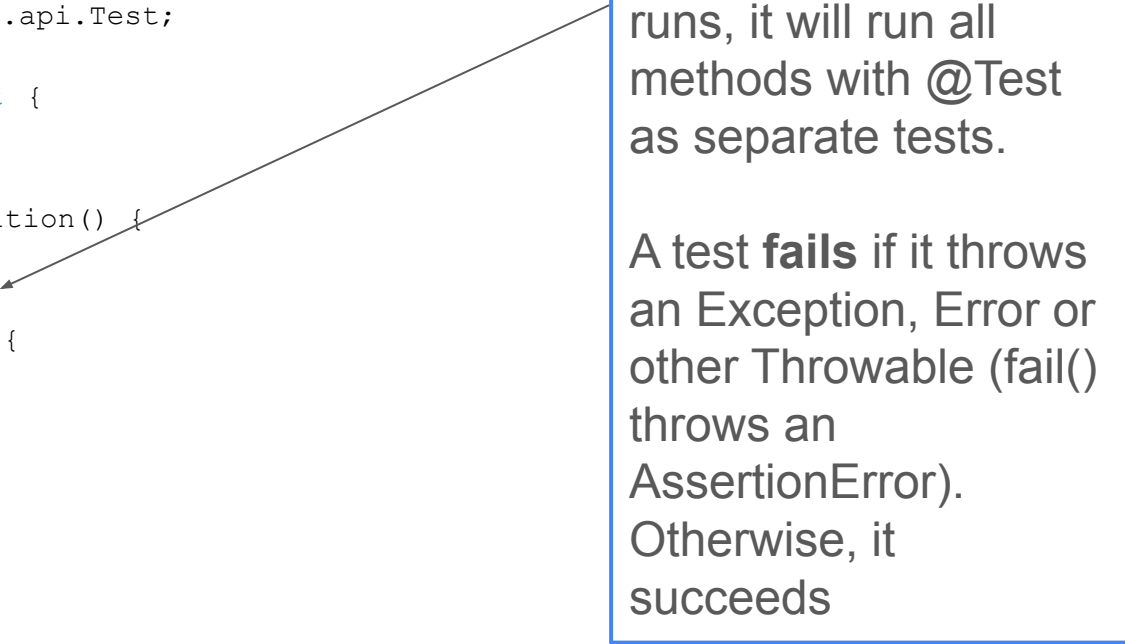
Writing Tests: JUnit

```
import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;

public class ExampleTest {

    @Test
    public void testAddition() {
        int a = 2;
        int b = 3;
        if (a + b != 5) {
            fail();
        }
    }
}
```



When a test class runs, it will run all methods with `@Test` as separate tests.

A test **fails** if it throws an Exception, Error or other Throwable (`fail()` throws an `AssertionError`). Otherwise, it succeeds

Your Turn!

You'll want a project for following along for today, so let's get that configured now

1. Follow the Testing Exercises link on Brightspace, and open with Github Desktop (or preferred git client)
2. From the command line in the repo folder, run `./gradlew eclipse`
3. Copy the ExampleTest class from the previous slide into a new file in a test folder in your repo (TestingExercises/test/ExampleTest.java)
4. Check that ExampleTest compiles and runs

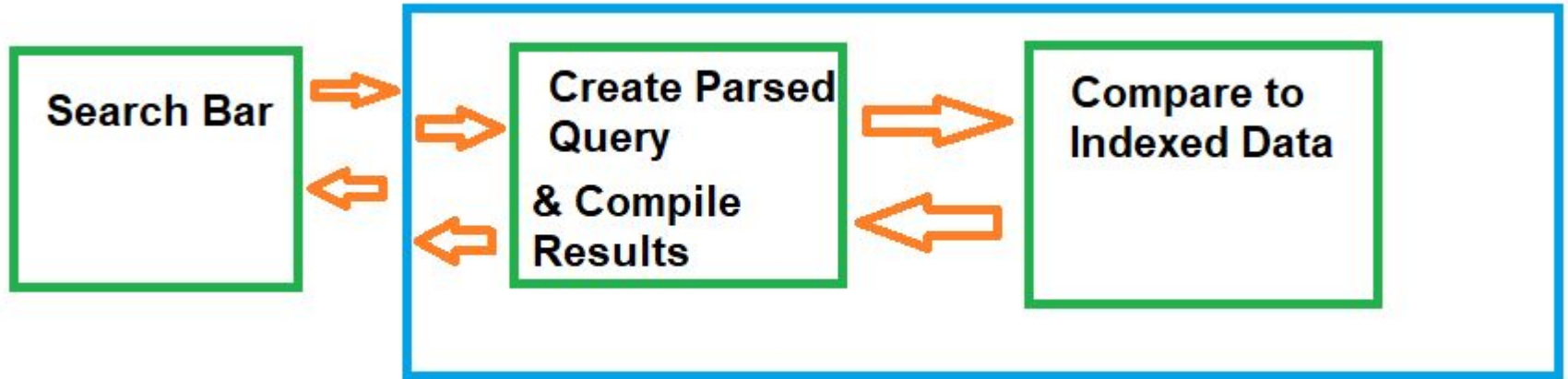
Note: you may need to change the junit-jupiter version in build.gradle, depending on what Eclipse version you have - you can get Eclipse to tell you what it expects (go through the steps of adding JUnit 5 as a library to your build path, but **don't actually add it**, just grab the version info on the penultimate step)

When To Use Which Type of Test

Remember the System Design Diagram from last week.

Suppose you write all the code, fire it up, and the search bar doesn't do anything.

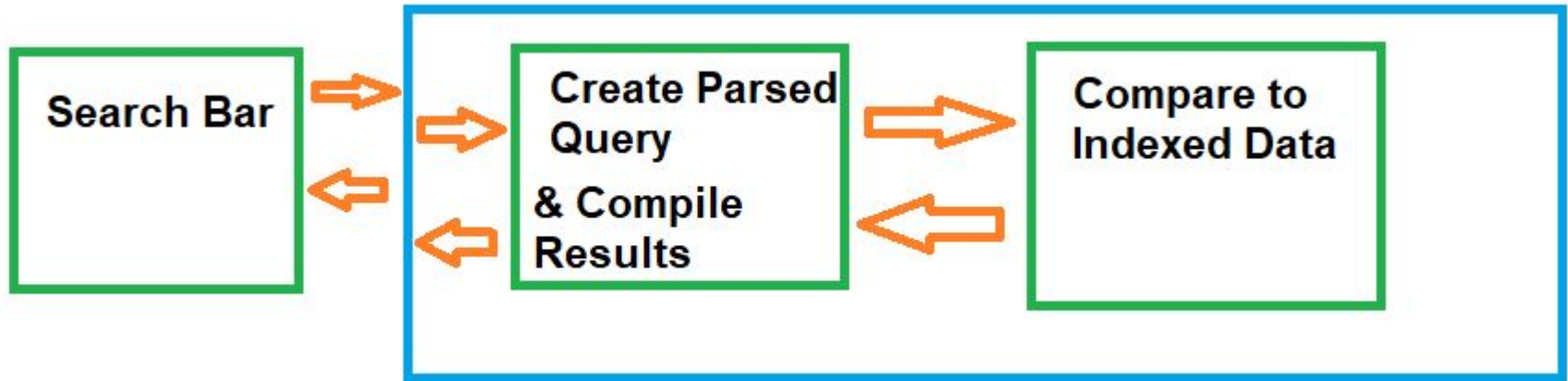
What's broken?



Unit Tests: Something is Wrong Inside a Box

Recall: a **unit test** tests just one component

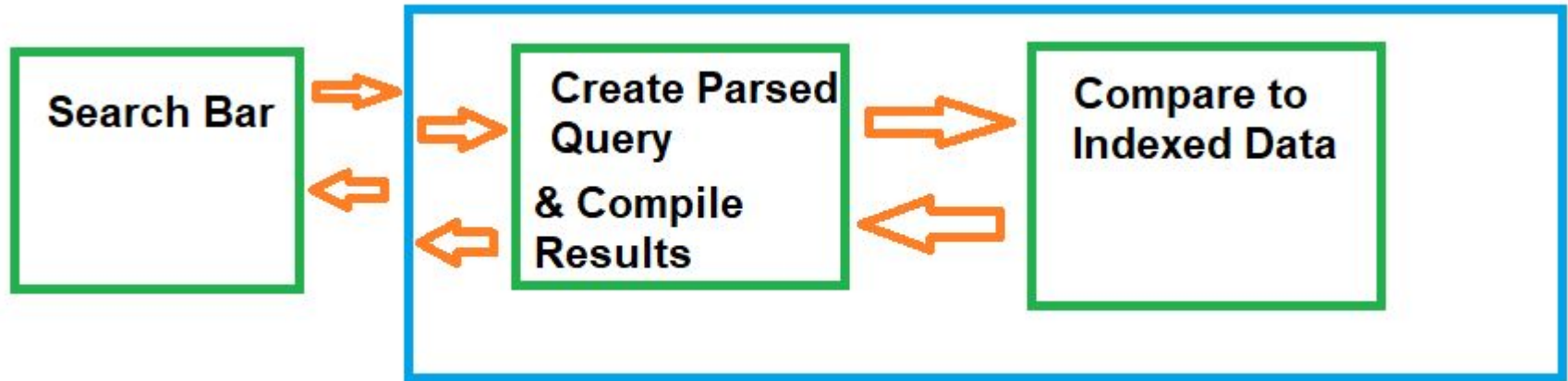
If something is wrong with **just** the search bar, a unit test for the search bar will fail



Integration Tests: Something is Wrong Between Boxes

Recall: an **integration test** tests multiple components (usually 2)

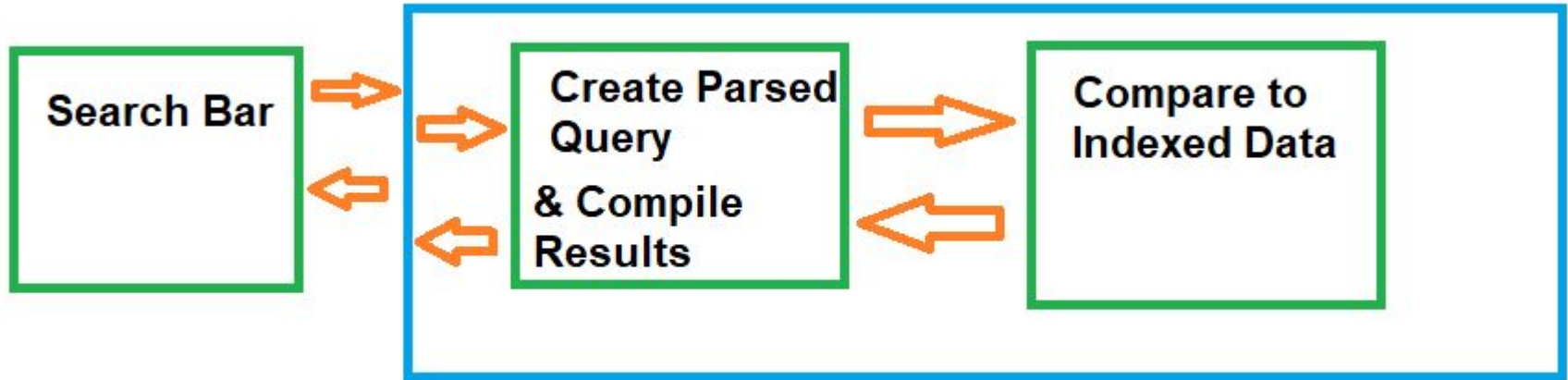
If something is wrong with sending a parsed query to the indexed data, that will cause an **integration test** to fail



End-To-End Tests: Something is Wrong Systematically

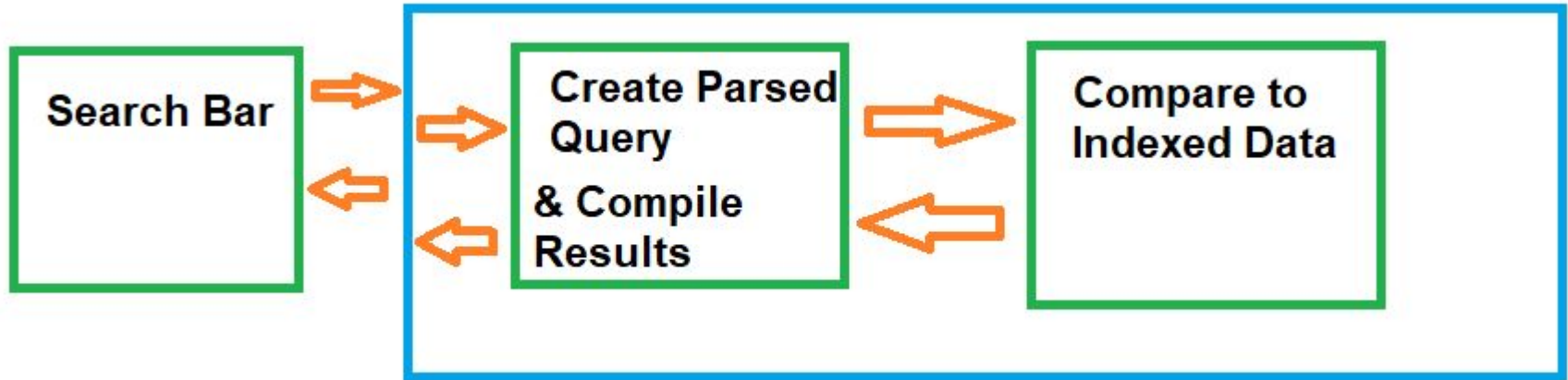
Recall: an **end-to-end test** checks an entire workflow

If all integration and unit tests are passing, an end-to-end failure will flag systemic issues (ex: SSL configuration mismatch)



Unit Tests: **Just** What's in One Box?

How to test that 'search results display correctly' while **only** using code from the 'Search Bar' component?



Mock Objects

In order to isolate just one component for testing, use **mock objects** for other components

- In-memory
- Dummy implementation (hardcoded return values for methods)
- No side effects
- Test-code only (often single-test only)

```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
              // guaranteed to be random.  
}
```

Mock Objects

```
import org.mockito.Mockito;  
import static org.mockito.Mockito.when;  
import static org.mockito.Mockito.any;
```

Common Framework: Mockito

Many other interesting methods that you can statically import!

Note: Most IDEs don't have good support for automatic static import detection; you may need to do this part by hand

Mock Objects

Creating the mock object: Mockito.mock()

- Interfaces
- Non-final classes

```
public void testComponent() throws Exception {  
    Database mockDatabase = Mockito.mock(Database.class);  
    when(mockDatabase.sqlQuery(any(String.class))).thenReturn(5);  
  
    AddingMachine testComponent = new AddingMachine(mockDatabase);  
    testComponent.add(2,3);  
}
```



Mock Objects

Dummy implementation: tell the mock how it should respond to requests

- Anything left unspecified will return default values and do nothing

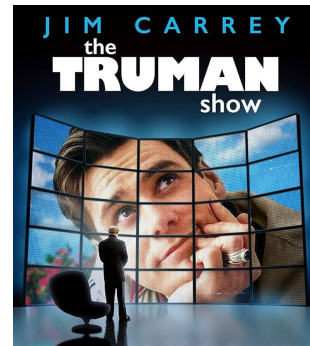
```
public static interface Database {  
    public int sqlQuery(String query);  
}  
  
public void testComponent() throws Exception {  
    Database mockDatabase = Mockito.mock(Database.class);  
    when(mockDatabase.sqlQuery(any(String.class))).thenReturn(5);  
  
    AddingMachine testComponent = new AddingMachine(mockDatabase);  
    testComponent.add(2,3);  
}
```



Mock Objects

Use the mock to create a **real version** of the component to test

Everything else except that one component is mocked



```
public void testComponent() throws Exception {  
    Database mockDatabase = Mockito.mock(Database.class);  
    when(mockDatabase.sqlQuery(any(String.class))).thenReturn(5);  
  
    AddingMachine testComponent = new AddingMachine(mockDatabase);  
    testComponent.add(2,3);  
}
```



Your Turn!

1. Look at the exercise1 package in the TestingExercises project
2. Within that, you'll find a class Widget and an interface Foo
3. Write a test for the Widget class that uses a mock Foo object, and verifies that calling addNumbers(2,3) does not throw an Exception

To the IDE!

Internet Advice Caution

Mockito.verify is often recommended (including in Mockito docs) online; don't use this for unit tests/smoke tests

- Annoying to configure
- Encourages bad testing practices (change-detection tests vs behavior tests)
- Good for some highly specific use cases (in particular, it's good for convincing devs with established code bases to migrate to Mockito)

Advantage of Mockito over other mock frameworks (EasyMock, JMock) is that you can easily avoid calling verify

- Look at the "stub method calls" examples instead

JUnit Assertions

```
public void testAddition() {  
    int result = new AddingMachine().add(2,3);  
    Assertions.assertEquals(5, result);  
}
```

JUnit: test fails $\Leftrightarrow$ test throws an Exception

Test logic: test fails $\Leftrightarrow$ some condition is not met

Assertions translates between the two: if the two values to assertEquals aren't equal, throws AssertionError

- Many, many methods available (arrayEquals, lessThan, greaterThan, etc)

Version note: in Junit 4, this class was called Assert. You can almost always directly replace Assert.<method> with Assertions.<method> to convert to Junit 5

Test Driven Development (TDD)

Write tests as you go along:

- Write the API (the **prototype** will often be the first test)
- Write some simple unit and integration tests (**smoke tests**)
- Write the **code**
- Write **bug-driven** tests (unit and integration)

Remember: a good test always needs to fail at first

Test Driven Development (TDD)

Smoke tests (where there's smoke there's fire):

Specific type of **unit test**

Check that all **basic** operations work for normal input

Does not need to handle **edge cases** or complex cases



Example:

```
public void testAddition() {  
    int result = new AddingMachine().add(2,3);  
  
    // old-school alternative to Assertions  
    assert result == 5;  
}
```

Failure Testing

Use tests to trigger unusual **error** states

- File system failures
- Networking failures
- More generally, any failure from another component's API



Failure Testing

For simple cases, use a **mock** object:

Mockito.thenThrow

```
Database mockDatabase = Mockito.mock(Database.class);  
when(mockDatabase.sqlQuery(any(String.class)))  
    .thenThrow(new RuntimeException("Database error"));
```

Failure Testing

But what if you need something more fully featured than a mock?

Add **testing hooks** in your code

- These are no-op methods designed for tests to override them to inject functionality

Combine these with a **test** implementation

- Fully functional (unlike a mock)
- Allows for failure testing in **integration** tests



Failure Testing

```
public class ServerConnection {
    public Connection getConnection() {
        testingHook();
        return connection;
    }

    protected void testingHook() {
        // does nothing, only implemented by tests
    }
}

public class ServerTestSuite {
    private boolean hasFailed = false;

    private static class TestServerConnection extends ServerConnection {
        protected void testingHook() {
            if (!hasFailed) {
                hasFailed = true;
                throw new ServerConnectionException();
            }
        }
    }
}
```

Bug-Driven Testing

Use tests to reproduce a bug in a controlled environment



Bug-Driven Testing

Happens **after** implementing code

Either actual reported bugs, or suspected bugs:

- Edge cases
- Unusual code paths

Combine with TDD smoke tests to get good **testing coverage**

Testing Coverage

Test Coverage is the percentage of branches and lines of code that are exercised by test code

Ideally, all lines of code would be tested; in practice, that's often excessive/not useful

Ex: an assert statement that never fails

Aim for **80-90%** code coverage

Testing Coverage Tools

Recommended: JaCoCo (<https://www.eclemma.org/jacoco/>)

- Originated from the Emma project (<https://emma.sourceforge.net/>), which is no longer well-maintained
- Includes IDE plugins for both IntelliJ and Eclipse
 - <https://plugins.jetbrains.com/plugin/103-emma-code-coverage>
 - <https://www.eclemma.org/>

Testing Coverage

Sample Coverage Output

```
@Test
public void testAddition() {
    int result = new AddingMachine().add(2,3);
    Assertions.assertEquals(5, result);
}
```

```
public class AddingMachine {

    private final String name;

    public AddingMachine(String name) {
        this.name = name;
    }

    public int add(int value1, int value2) {
        if (value1 < value2) {
            return value1 + value2;
        }
        throw new IllegalStateException("This is definitely a bug");
    }

    public String getName() {
        return name;
    }
}
```

Element	Coverage	Covered Instru...	Missed Instruct...	Total Instructio...
▼ DebugExamples	9.1 %	30	301	331
▼ src	9.1 %	30	301	331
▼ (default package)	9.1 %	30	301	331
> TestAddingMachine.java	81.0 %	17	4	21
> AddingMachine.java	61.9 %	13	8	21

Using Test Coverage Output

80% coverage is fine

Focus on quality of tests more than coverage numbers

Use the output to spot problem areas/missing testing



Fuzz Testing

A version of automated **exploratory** testing

Uses randomization to test out unexpected paths



Randomization in tests must be done carefully, or the tests stop being useful:

- For each test run, print the **seed** used for the random number generator
- Allow a **manual** run to specify a seed

This lets you have **repeatable** tests, which is critical for identifying and fixing bugs

Fuzz Testing Example

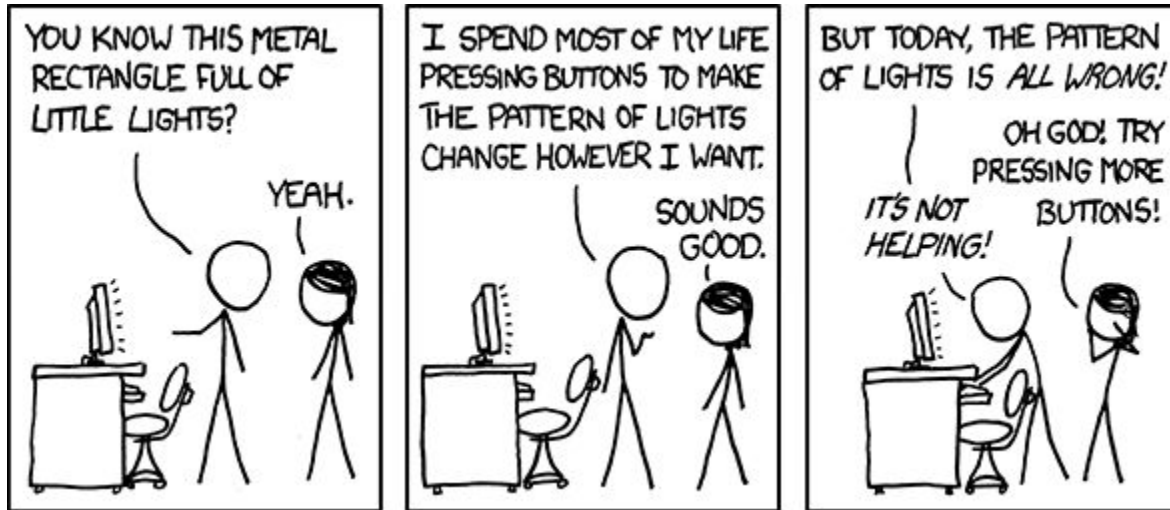
```
public class TestWidget {  
  
    private static int NUM_TESTS_PER_RUN = 100;  
  
    @Test  
    public void testFuzzy() {  
        long seed = System.currentTimeMillis();  
        Random random = new Random(seed);  
        Widget testWidget = new Widget();  
        runFuzzyTest(seed, random, testWidget);  
    }  
  
    private static void runFuzzyTest(long seed, Random random, Widget testWidget) {  
        System.out.println("Running test with seed " + seed);  
        for (int i = 0; i < NUM_TESTS_PER_RUN; i++) {  
            int a = random.nextInt();  
            Assert.assertEquals(computeSlow(a), testWidget.compute(a));  
        }  
    }  
  
    public static void main(String[] args) {  
        long seed = Long.parseLong(args[0]);  
        Random random = new Random(seed);  
        Widget testWidget = new Widget();  
        runFuzzyTest(seed, random, testWidget);  
    }  
}
```

Your Turn!

1. Check out the exercise2 package in TestingExercises
2. Run the tests, check that the basic **smoke test** passes
3. Using the checkComputation method, write a fuzzy test that will add lots of different random integers
4. Using the fuzzy test, find an input that triggers the bug in AddingMachine.java

Software Engineering

Debugging



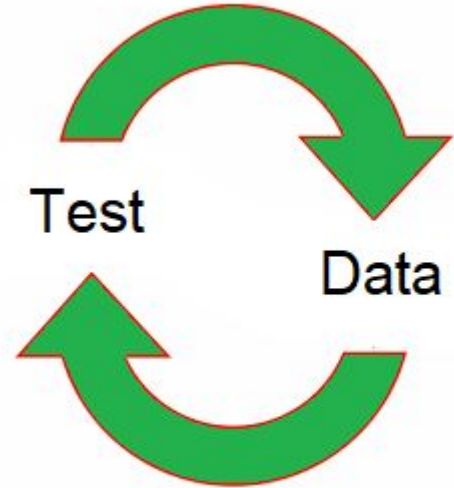
Debugging Philosophy

Quick iteration is key

Prefer **many** tests that each check one small feature

Code complexity is **quadratic** with lines of code

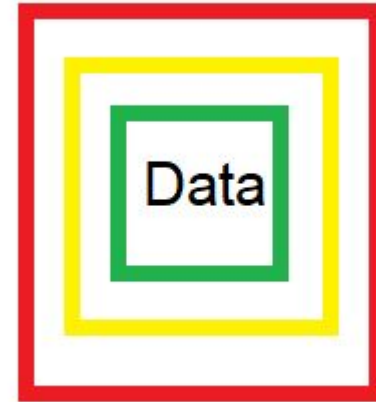
Finding a bug is a **search** problem



Design for Debugability

Keep state as local as possible

- Local variables
- Parameters
- Private data
- Public data



Design for Debugability

Make shared data immutable wherever possible

Example: Builder pattern

```
WidgetBuilder newWidget = new WidgetBuilder();  
newWidget.setSpinnerSpeed(getSpinnerSpeed());  
newWidget.setNumberOfGears(numGears);  
return newWidget.build();
```

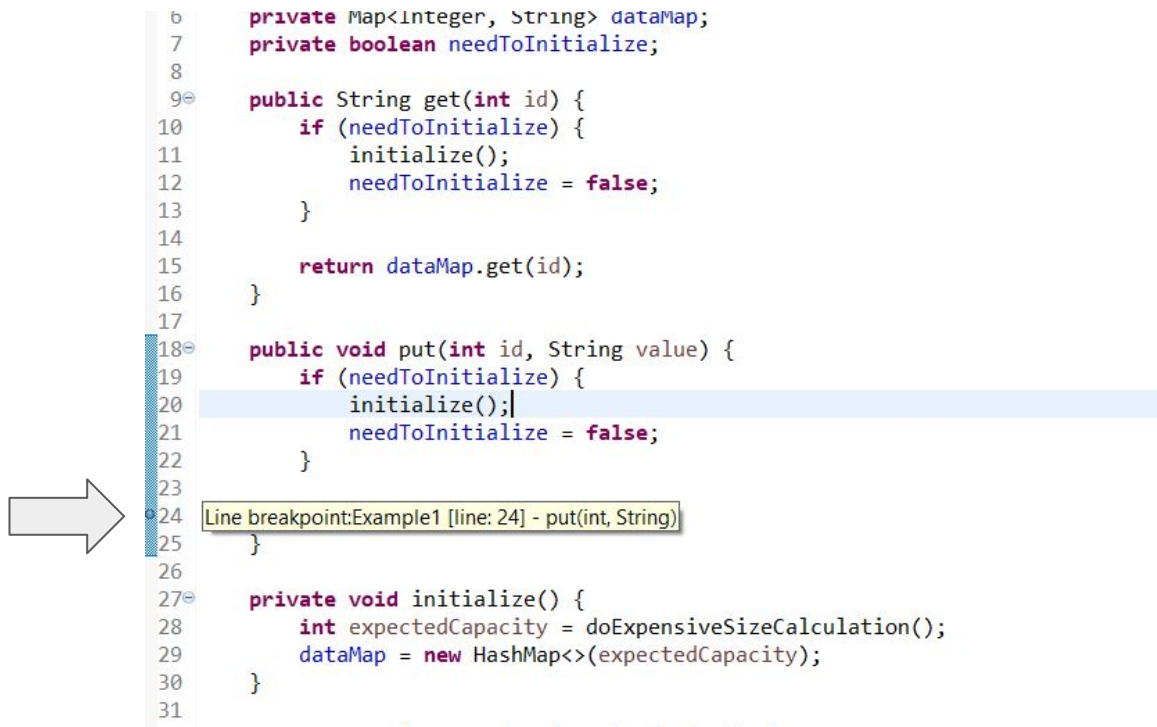
Design for Debugability

Create wrapper classes to keep code logic in one file

```
public class WidgetCache {  
    private Map<Long, Widget> idToWidgetCache = new HashMap<>();  
  
    public Widget put(long id, Widget widget) {  
        return idToWidgetCache.put(id, widget);  
    }  
  
    public Widget get(long id) {  
        return idToWidgetCache.get(id);  
    }  
  
    public void clear() {  
        idToWidgetCache.clear();  
    }  
}
```

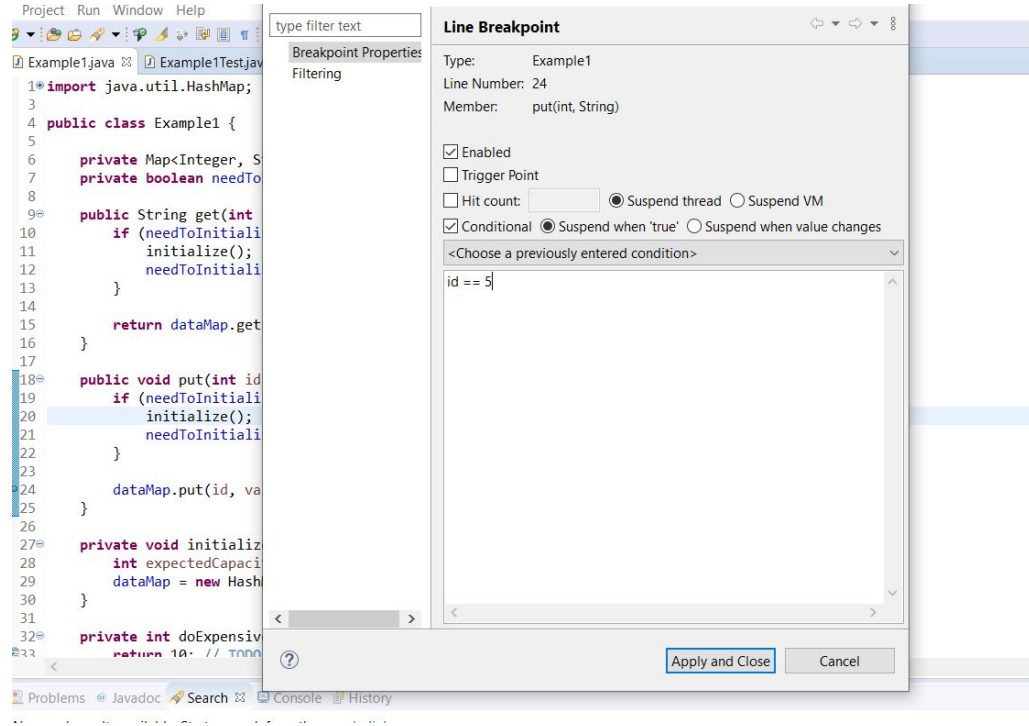
Useful IDE Tools

Breakpoints



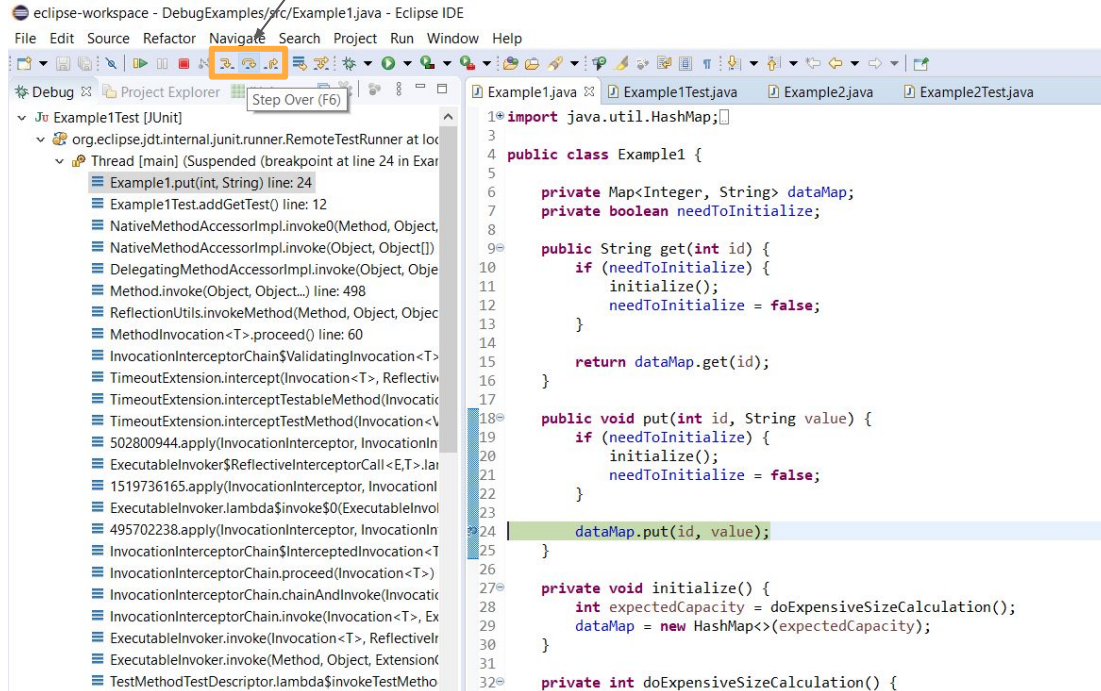
Useful IDE Tools

Conditional Breakpoints



Useful IDE Tools

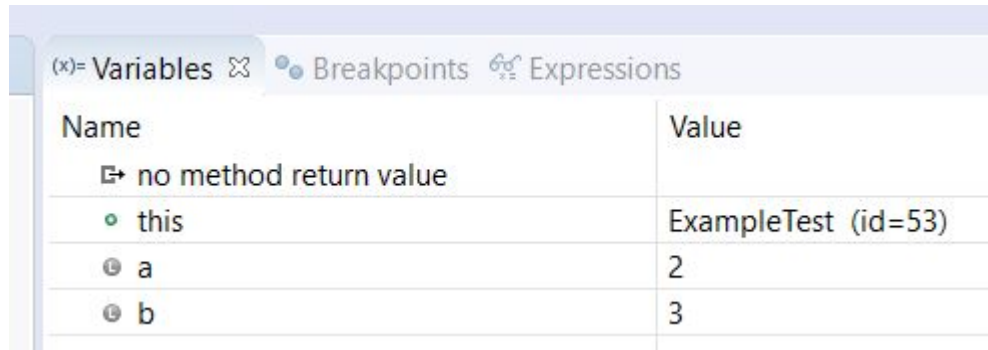
Control Flow: Step in/out/over



Useful IDE Tools

Tons of useful info available in the debugger

Much faster to examine live vs. println debugging



A screenshot of the 'Variables' window in an IDE. The window has tabs for '(x)= Variables', 'Breakpoints', and 'Expressions'. The 'Variables' tab is active, showing a table of variables. The table has two columns: 'Name' and 'Value'. The rows are: 'no method return value' (with a small icon), 'this' (with a green dot icon), 'a' (with a small icon), and 'b' (with a small icon). The value for 'this' is 'ExampleTest (id=53)', for 'a' is '2', and for 'b' is '3'.

Name	Value
no method return value	
this	ExampleTest (id=53)
a	2
b	3

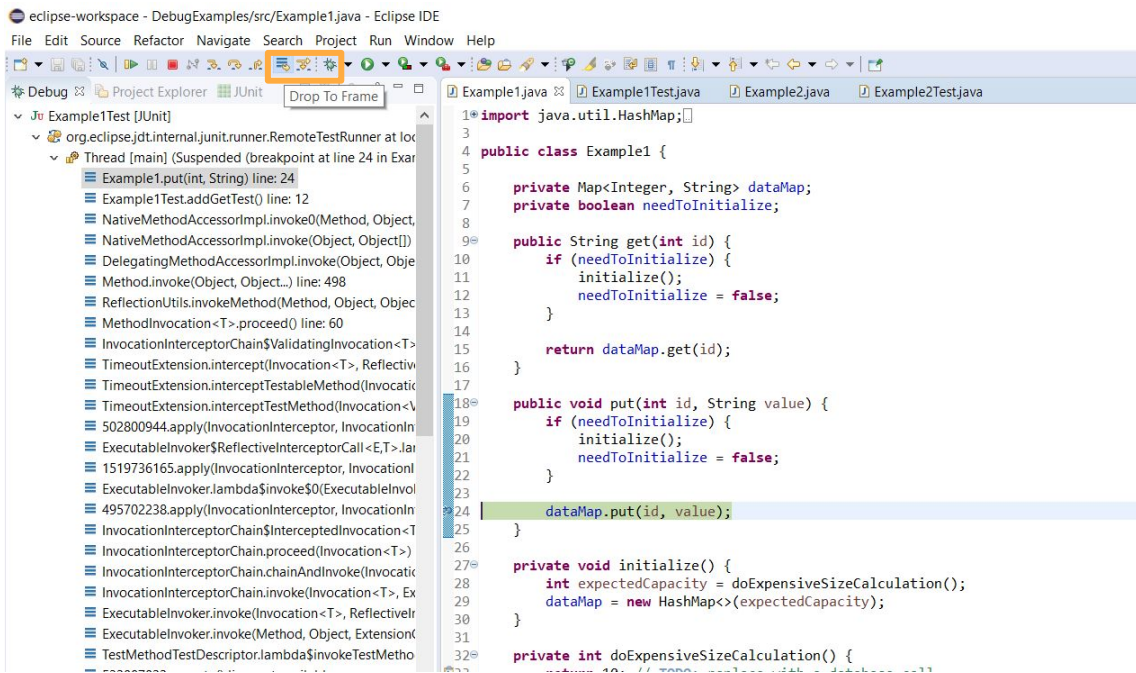


A screenshot of the 'Expressions' window in an IDE. The window has tabs for '(x)= Variables', 'Breakpoints', and 'Expressions'. The 'Expressions' tab is active, showing a table of expressions. The table has two columns: 'Name' and 'Value'. The rows are: 'x+y "a + b"' (with a small icon) and 'Add new expression' (with a green plus icon). The value for 'x+y "a + b"' is '5'.

Name	Value
x+y "a + b"	5
+ Add new expression	

Useful IDE Tools

Drop To Frame (Caution: Does not reset **non-local** state, such as fields)



Debugging Example

Build a smoke test first

```
public class WrappedHashMapTest {  
  
    @Test  
    public void addGetTest() {  
        int id = 5;  
        String value = "five";  
        WrappedHashMap example = new WrappedHashMap();  
        example.put(id, value);  
        Assert.assertEquals("five", example.get(5));  
    }  
}
```

Debugging Example

Next, write the implementation

We're leaving the cross-component db call unimplemented for the moment

Because we'll have that eventually, though, we need to use **lazy initialization**

Note that you should never rely on a user to initialize!

```
public class WrappedHashMap {  
    private Map<Integer, String> data;  
    private boolean needToInitialize;  
  
    public String get(int id) {  
        if (needToInitialize) {  
            initialize();  
            needToInitialize = false;  
        }  
  
        return data.get(id);  
    }  
  
    public void put(int id, String value) {  
        if (needToInitialize) {  
            initialize();  
            needToInitialize = false;  
        }  
  
        data.put(id, value);  
    }  
  
    private void initialize() {  
        int initialCapacity = 10;  
        data = new HashMap<>(initialCapacity);  
    }  
}
```

Debugging Example

To the debugger!

Debugging Example

Recap of the process:

- 1) Put a breakpoint on the failing line
- 2) Run through a test that triggers the failure, and check instance and local variable state at the breakpoint
- 3) Add a breakpoint where the value of an incorrect variable should have changed
- 4) Repeat

Your Turn!

1. Check out the exercise3 package in TestingExercises
2. Run the TestCombinatorics test; it should fail
3. Put a breakpoint at the last line in the stack trace, and see what variables are invalid at that line
4. Find where the invalid variable should have been set, put another breakpoint there (for method return values, use the 'return' line of the method)
5. Repeat this process - for breakpoints within for loops, use a conditional breakpoint to only stop the breakpoint when something interesting is happening
6. Fix the bug, and verify the test is now green

Let's Review (Testing & Debugging)

- Two main **categories** of testing: manual vs automated, what each is good for
- Three types of **regression** testing
- What is a **mock** object and what is it used for
- What is a **testing hook** and what is it used for
- What is **fuzz testing** (...and what is it used for)
- What are three ways to **design for debugability** in your code